SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	N Goutham	**Reg. No.:**	192472024
Section / Batch:	CSE-AI	**Date:**	07-08-2026

Code of Conduct

I N Goutham/192472024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N Goutham

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
- GitHub Repository Link in GCR
- Google Colab Share Link in GCR
- Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject–verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

S → NP VP
NP → Det N
VP → V NP

Det → the | a
N → student | teacher | book
V → reads | likes

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
    pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

Aim:

To design a Python program that validates whether a sentence follows the given Context-Free Grammar (CFG) rules and generates its parse tree representation.

Problem Statement:

Design a Python program to:

1. Accept a sentence as input.
2. Verify whether it conforms to the given CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Grammar Rules:

S → NP VP
NP → Det N
VP → V NP

Det → the | a
N → student | teacher | book
V → reads | likes

Algorithm:

1. Define the CFG grammar rules.
2. Accept a sentence from the user.
3. Tokenize the sentence into words.
4. Parse the sentence using CFG.
5. If parsing succeeds:

   Display "Valid Sentence".

   Generate and display the parse tree.

6. Otherwise:

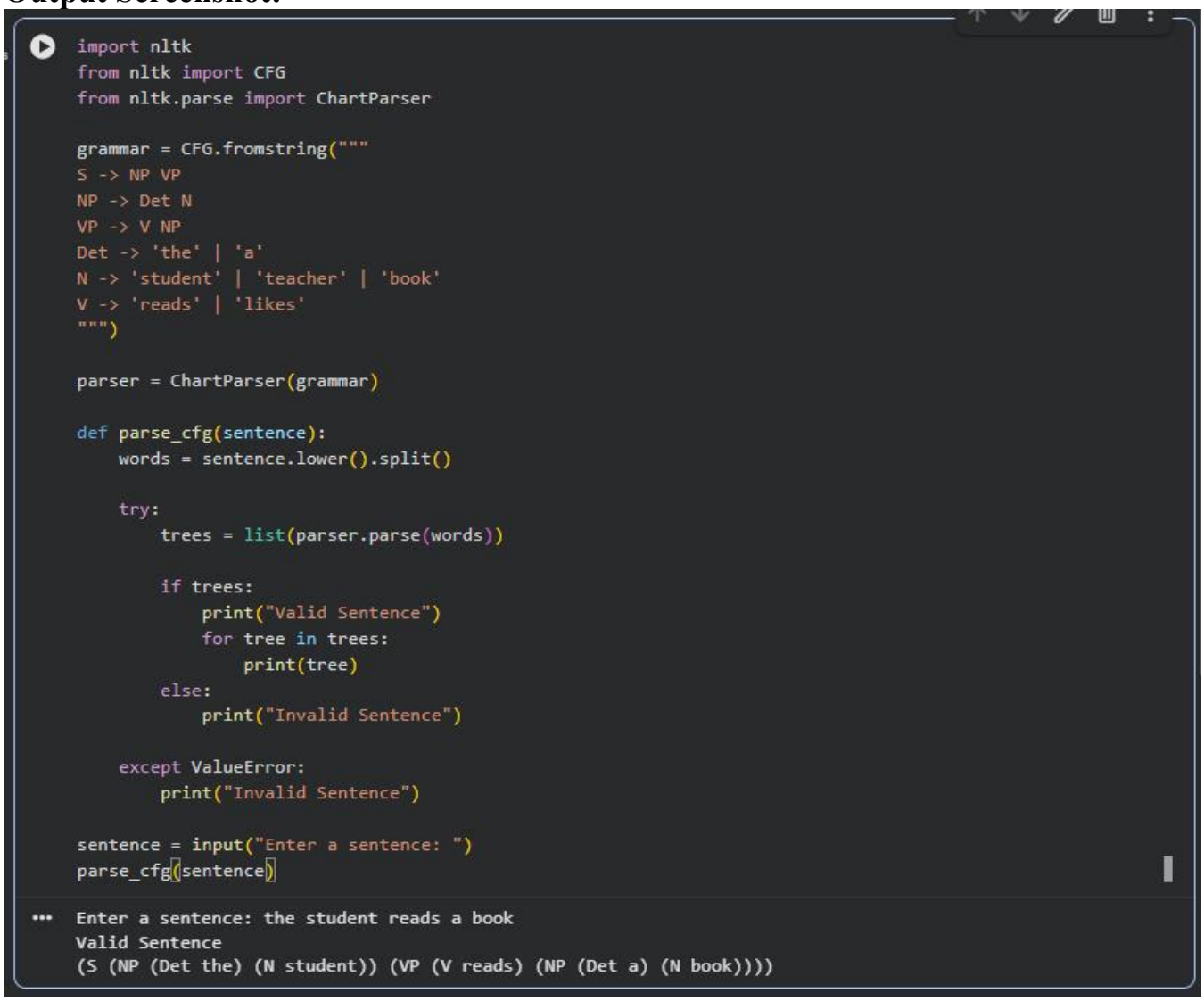
   1. Display "Invalid Sentence".

Python Code:

```
import nltk
from nltk import CFG
from nltk.parse import ChartParser

grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'a'
N -> 'student' | 'teacher' | 'book'
V -> 'reads' | 'likes'
""")

parser = ChartParser(grammar)

def parse_cfg(sentence):
    words = sentence.lower().split()

    try:
        trees = list(parser.parse(words))

        if trees:
            print("Valid Sentence")
            for tree in trees:
                print(tree)
        else:
            print("Invalid Sentence")

    except ValueError:
        print("Invalid Sentence")

sentence = input("Enter a sentence: ")
```

parse_cfg(sentence)

Output Screenshot:



```
import nltk
from nltk import CFG
from nltk.parse import ChartParser

grammar = CFG.fromstring("""
S -> NP VP
NP -> Det N
VP -> V NP
Det -> 'the' | 'a'
N -> 'student' | 'teacher' | 'book'
V -> 'reads' | 'likes'
""")

parser = ChartParser(grammar)

def parse_cfg(sentence):
    words = sentence.lower().split()

    try:
        trees = list(parser.parse(words))

        if trees:
            print("Valid Sentence")
            for tree in trees:
                print(tree)
        else:
            print("Invalid Sentence")

    except ValueError:
        print("Invalid Sentence")

sentence = input("Enter a sentence: ")
parse_cfg(sentence)
```

```
Enter a sentence: the student reads a book
Valid Sentence
(S (NP (Det the) (N student)) (VP (V reads) (NP (Det a) (N book))))
```

Test Cases and Output:

Test Case 1

Input:

the student reads a book

Output

Valid Sentence

(S
  (NP (Det the) (N student))
  (VP (V reads)
    (NP (Det a) (N book))))

Test Case 2

Input

a teacher likes the book

Output

Valid Sentence

```
(S
  (NP (Det a) (N teacher))
  (VP (V likes)
    (NP (Det the) (N book))))
```

Test Case 3

Input

student reads book

Output

Invalid Sentence

Test Case 4

Input

the book likes a teacher

Output

Valid Sentence

```
(S
  (NP (Det the) (N book))
  (VP (V likes)
    (NP (Det a) (N teacher))))
```

Test Case 5

Input

reads the student book

Output

Invalid Sentence

These outputs correspond to the test cases specified in the assessment.

Result Analysis:

- Sentences following the CFG structure S → NP VP are successfully parsed.
- Parse trees clearly represent the grammatical structure.
- Sentences violating CFG rules are correctly identified as invalid.
- The program effectively validates English sentence syntax using Context-Free Grammar.

Result:

The Python program successfully validates sentences using the given CFG rules and generates parse tree representations for valid sentences while rejecting invalid sentences. This demonstrates the application of CFG and parsing techniques in Natural Language Processing.

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

he → singular, third person
she → singular, third person
it → singular, third person
they → plural

runs → singular verb
writes → singular verb

run → plural verb
write → plural verb

Task

1. Accept Subject and Verb.
2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject	Verb	Expected
he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Aim

To design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes and validates grammatical constraints.

Problem Statement:

Design a Python program that:

1. Accepts a Subject and a Verb.
2. Compares their feature structures.
3. Returns True if agreement holds.

4. Returns False otherwise.

Feature Rules

Subject	Feature Structure
he	singular, third person
she	singular, third person
it	singular, third person
they	plural

Verb	Feature Structure
runs	singular verb
writes	singular verb
run	plural verb
write	plural verb

Algorithm:

1. Define feature structures for subjects and verbs.
2. Accept subject and verb as input.
3. Extract Number and Person features.
4. Compare subject features with verb features.
5. If features match:
    1. Return True
6. Otherwise:
    1. Return False
7. Display the result.

Python Code:

```
def check_agreement(subject, verb):

    subject_features = {
        "he": ("singular", "third"),
        "she": ("singular", "third"),
        "it": ("singular", "third"),
        "they": ("plural", "third")
    }

    verb_features = {
        "runs": "singular",
        "writes": "singular",
        "run": "plural",
        "write": "plural"
    }

    if subject not in subject_features or verb not in verb_features:
```

```
    return False

subject_number = subject_features[subject][0]
verb_number = verb_features[verb]

return subject_number == verb_number

subject = input("Enter Subject: ").lower()verb = input("Enter Verb: ").lower()
print(check_agreement(subject, verb))
```

Output Screenshot:

Experimental Design

Feature Structure Representation

Subject
↓
(Number, Person)
↓
Feature Matching
↓
Verb Features
↓
Agreement Check
↓
True / False

Test Cases and Outputs

Test Case 1

Input : he runs

Output

True

Test Case 2

Input : he run

Output

False

Test Case 3

Input : they run

Output

True

Test Case 4

Input : they writes

Output

False

Test Case 5

Input : she writes

Output:

True

These outputs match the expected test cases provided in the assessment document.

Result Analysis:

- Singular subjects (he, she, it) correctly match singular verbs (runs, writes).
- Plural subjects (they) correctly match plural verbs (run, write).
- Mismatched Number features are identified and rejected.
- The feature structure approach effectively validates grammatical constraints and subject–verb agreement.

Result:

The Python program successfully verifies subject–verb agreement using feature structures based on Number and Person attributes. It correctly returns True for grammatically valid combinations and False for invalid combinations, demonstrating the application of feature structures and subcategorization rules in Natural Language Processing.

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

S → NP VP [1.0]

NP → John [0.6]
NP → Mary [0.4]

VP → runs [0.5]
VP → walks [0.5]

Probability Formula

P(Sentence)
=
P(S→NP VP)
× P(NP)
× P(VP)

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
    pass
```

Test Cases

Input Sentence	Expected Probability
John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Aim:

To design a Python program that parses sentences and computes sentence probabilities using Probabilistic Context-Free Grammar (PCFG) production rules.

Problem Statement:

Design a Python program to:

1. Accept a two-word sentence.
2. Parse the sentence using the given PCFG.

3. Compute the sentence probability.
4. Return **0** if the sentence cannot be generated by the grammar.

PCFG Grammar:

S → NP VP [1.0]

NP → John [0.6]
NP → Mary [0.4]

VP → runs [0.5]
VP → walks [0.5]
Probability Formula
P(Sentence)

= P(S → NP VP)
× P(NP)
× P(VP)

Algorithm

1. Define the probabilities of NP and VP productions.
2. Accept a sentence from the user.
3. Split the sentence into NP and VP.
4. Check whether the words exist in the grammar.
5. Compute the probability using:

P(Sentence) =P(S→NP VP) × P(NP) × P(VP)

1. If any word is not present in the grammar:

    1. Return 0.00

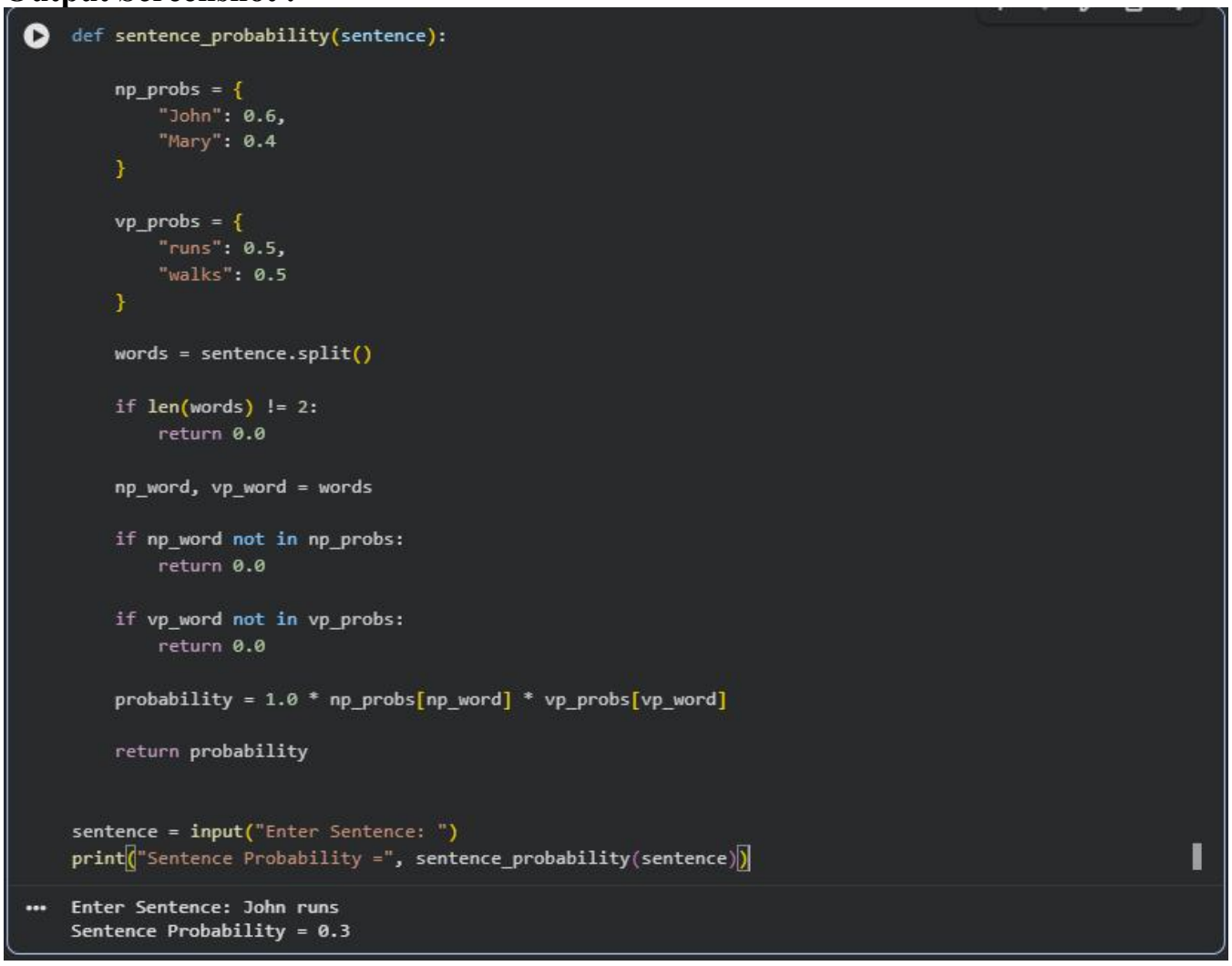
2. Display the final probability.

Python Code:

```
def sentence_probability(sentence):

    np_probs = {
        "John": 0.6,
        "Mary": 0.4
    }

    vp_probs = {
        "runs": 0.5,
        "walks": 0.5
    }

    words = sentence.split()

    if len(words) != 2:
        return 0.0
```

```
    np_word, vp_word = words

    if np_word not in np_probs:
        return 0.0

    if vp_word not in vp_probs:
        return 0.0

    probability = 1.0 * np_probs[np_word] * vp_probs[vp_word]

    return probability

sentence = input("Enter Sentence: ")print("Sentence Probability =", sentence_probability(sentence))
```

Output Screenshot :



Experimental Design:

PCFG Parsing Process

Input Sentence

↓

Split into NP and VP

↓

Check Grammar Rules

↓

Retrieve Rule Probabilities

↓

Apply PCFG Formula

↓

Sentence Probability

Test Cases and Outputs

Test Case 1

Input : John runs

Calculation

$P = 1.0 \times 0.6 \times 0.5$
$= 0.30$

Output

Sentence Probability = 0.30

Test Case 2

Input : John walks

Calculation

$P = 1.0 \times 0.6 \times 0.5$
$= 0.30$

Output

Sentence Probability = 0.30

Test Case 3

Input : Mary runs

Calculation

$P = 1.0 \times 0.4 \times 0.5$
$= 0.20$

Output

Sentence Probability = 0.20

Test Case 4

Input : Mary walks

Calculation

$P = 1.0 \times 0.4 \times 0.5$
$= 0.20$

Output

Sentence Probability = 0.20

Test Case 5

Input : Peter runs

Calculation

Peter is not present in NP rules

Output

Sentence Probability = 0.00

These outputs match the expected results given in the assessment document.

Result Analysis:

- The PCFG parser successfully computes sentence probabilities using grammar production rules.
- Sentences generated by the grammar receive valid probabilities.
- Sentences containing words outside the grammar receive a probability of **0.00**.
- Higher probability values indicate more likely sentence constructions according to the grammar.
- The approach demonstrates how probabilistic grammars are used in NLP parsing and language modeling.

Result:

The Python program successfully parses valid sentences and computes their probabilities using the given Probabilistic Context-Free Grammar. Invalid sentences that cannot be generated by the grammar are correctly assigned a probability of **0.00**. This demonstrates the practical application of PCFGs for probabilistic sentence parsing and syntactic analysis in Natural Language Processing.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results

Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____